Analysis and Design of Algorithms

NUS National University of Singapore

CS3230

Week 4

Sorting lower bound

Linear-time sorting

Ken Sung

Diptarka Chakraborty

The sorting problem

- ***Input:*** sequence $\langle a_1, a_2, \ldots, a_n \rangle$ of numbers.
- ***Output:*** permutation $\langle a'_1, a'_2, \ldots, a'_n \rangle$ such that $a'_1 \leq a'_2 \leq \cdots \leq a'_n$.

- Example:
  - ***Input:*** 8 2 4 9 3 6
  - ***Output:*** 2 3 4 6 8 9

- We learned a number of sorting algorithms:
  - Bubble sort, Insertion sort, Merge sort, Heap sort, Quick sort, etc.

Classification of the sorting algorithm

- The sorting algorithms are classified by
  - Running time (i.e. $O(n^2)$, $O(n \log n)$)

- We can also classify the sorting algorithms by
  1. In-Place
  2. Stable
  3. Comparison sort or not

In-Place

- A sorting algorithm is in-place if it uses very little additional memory beyond that used for the data, usually $O(1)$ or $O(\lg n)$.

- Useful when memory is limited.

Examples:

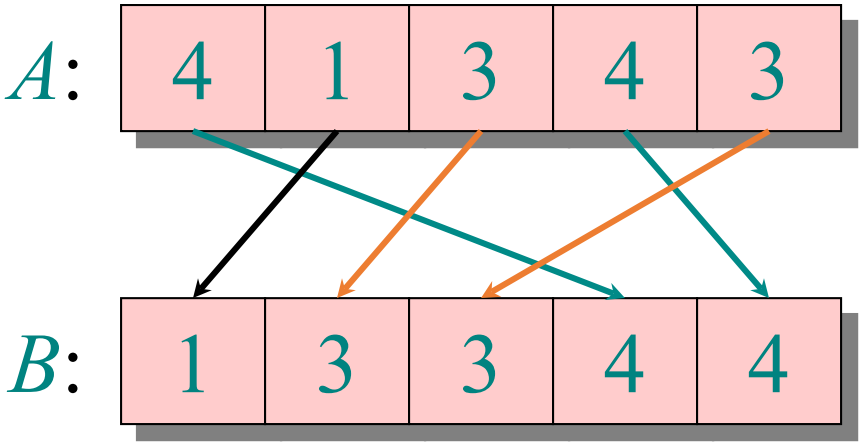
- Insertion sort
- Quicksort ($O(\lg n)$ additional memory with proper implementations)

Stable

- A sorting algorithm is stable if the original order of equal elements is preserved in the sorted output.

Examples:

- Insertion sort
- Merge sort



Comparison-based algorithm

- Comparison-based algorithm is an algorithm that only compares elements.

- A sorting algorithm that sort the elements by comparing them only is called a comparison sort.

- All the sorting algorithms we have seen so far are ***comparison sorts***: only use comparisons to determine the relative order of elements.
  - *E.g.*, insertion sort, merge sort, quicksort, heapsort.

What we want to study today?

1. What is the best time complexity for a comparison sort?

2. Is there any sorting algorithm which is better than a comparison sort?

What is the best time complexity for a comparison sort?

- So far, all sorting algorithms we learned are comparison sorts.

- The best worst-case running time that we've seen for comparison sorting is $O(n \lg n)$.

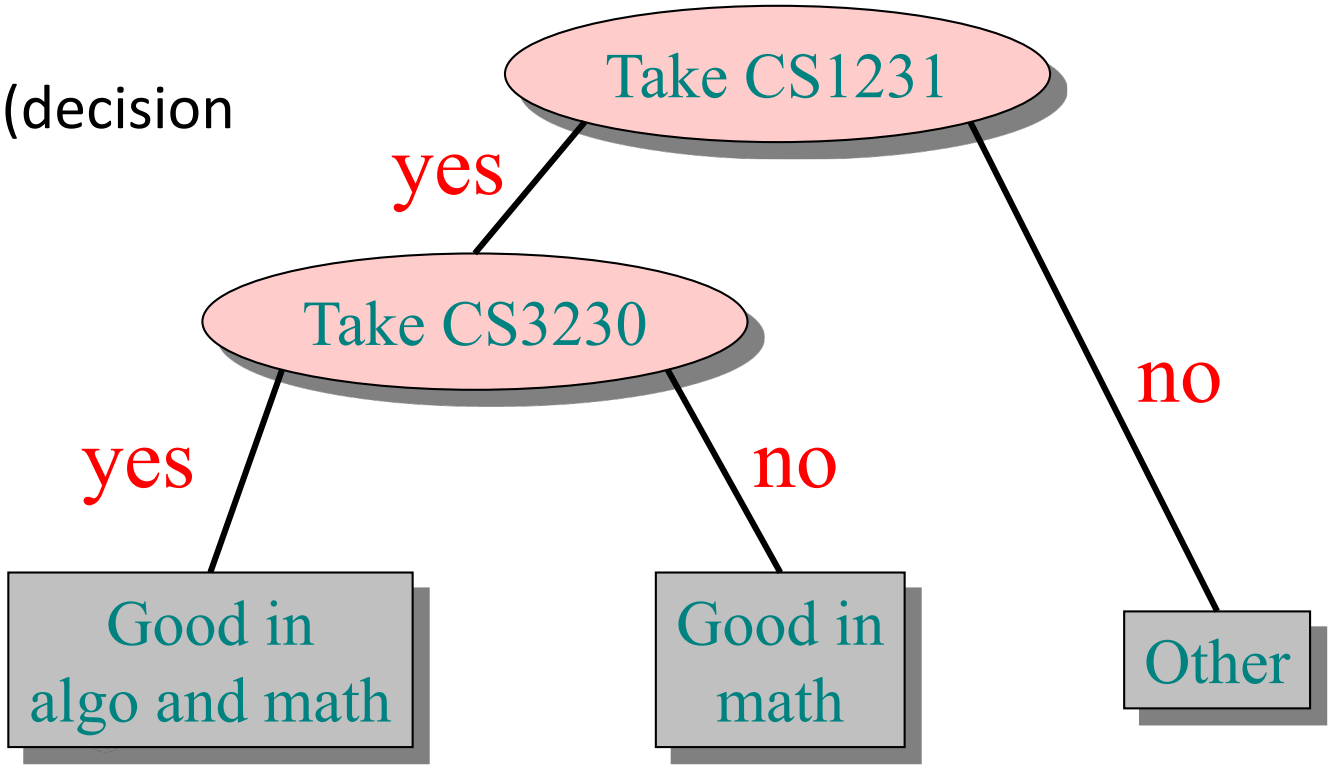
- ***Is $O(n \lg n)$ the best we can do?***
  - ***Decision trees*** can help us answer this question.

What is a decision tree?

```
if (take CS1231) then
    if  (take CS3230) then
        return "Good in Algo & Math"
    else
        return "Good in Math"
else
    return "other"
```

- A decision tree is a tree-like model.
  - Every node is a comparison.
  - Every branch represents the outcome of the comparison
  - Every leaf represents a class label (decision after all comparisons)

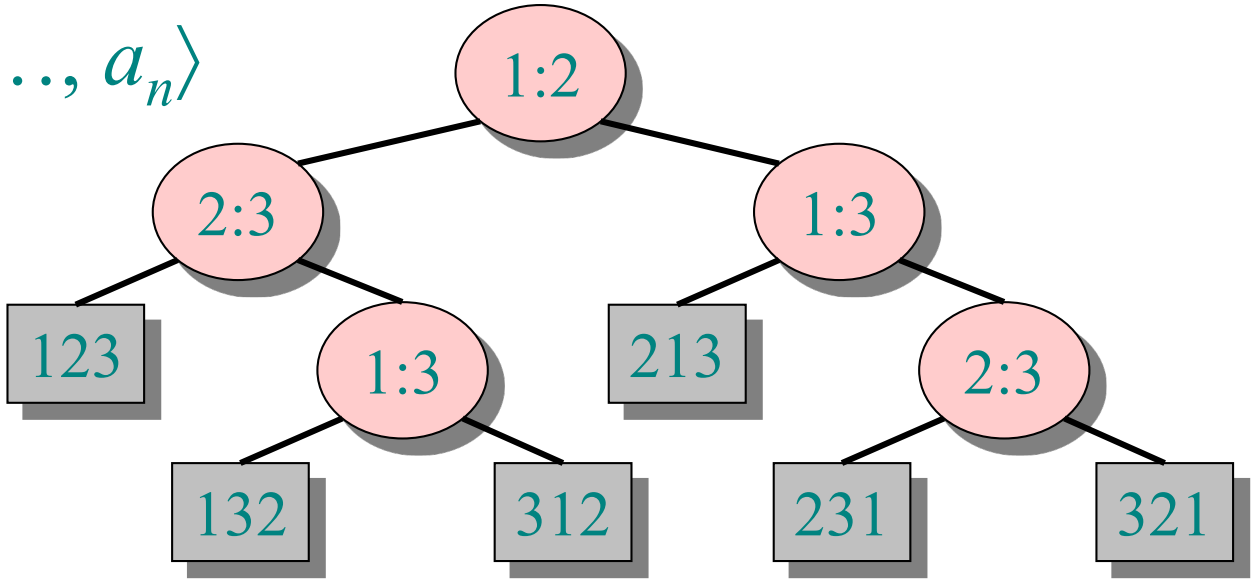
- We can use decision tree to model any comparison-based algorithm.



Decision-tree example

A decision tree models the execution of any comparison sort.

Sort $\langle a_1, a_2, \ldots, a_n \rangle$

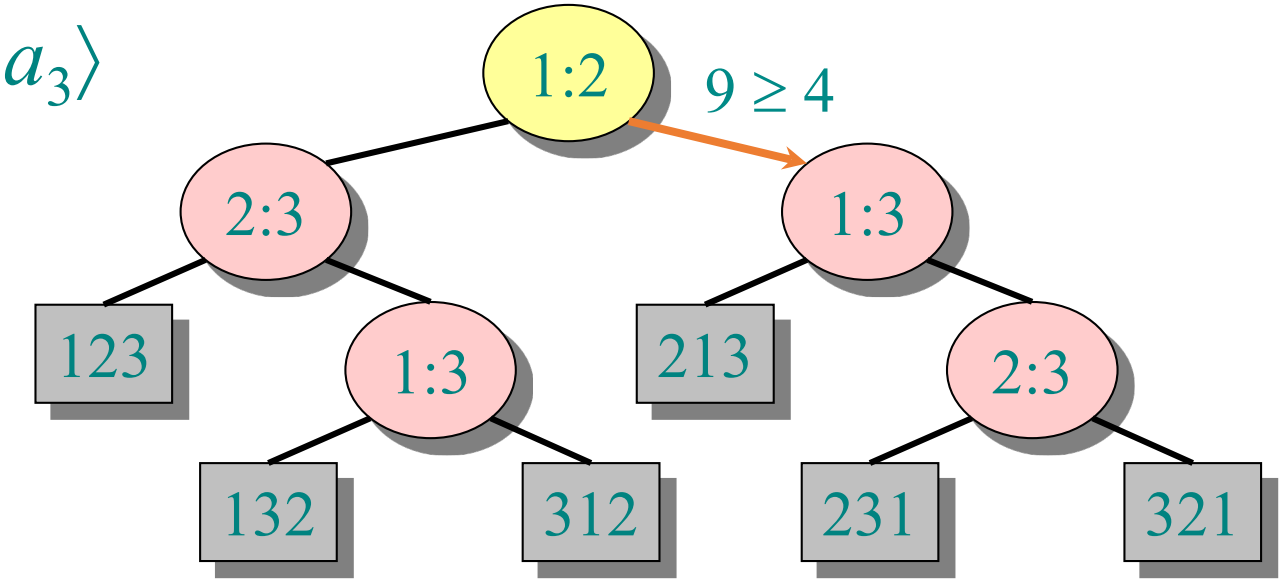


Each internal node is labeled $i{:}j$ for $i, j \in \{1, 2, \ldots, n\}$.

- The left subtree shows subsequent comparisons if $a_i < a_j$.
- The right subtree shows subsequent comparisons if $a_i \geq a_j$.

Decision-tree example

Sort $\langle a_1, a_2, a_3 \rangle$
$= \langle\ 9, 4, 6\ \rangle$:

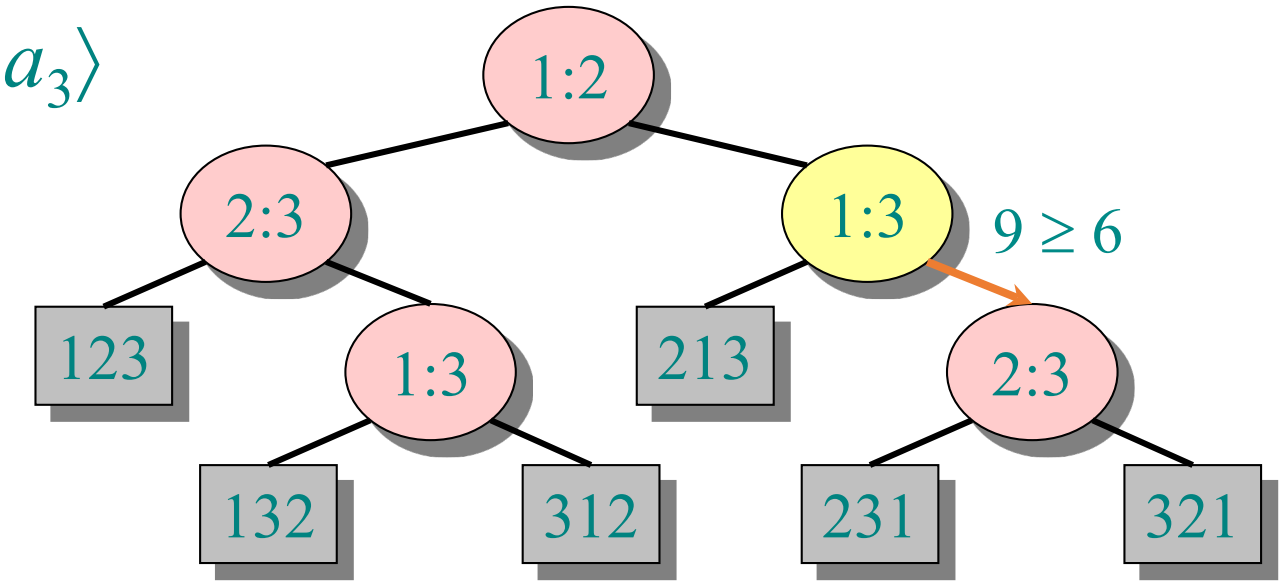


Each internal node is labeled $i{:}j$ for $i, j \in \{1, 2, \ldots, n\}$.

- The left subtree shows subsequent comparisons if $a_i < a_j$.
- The right subtree shows subsequent comparisons if $a_i \geq a_j$.

Decision-tree example

Sort $\langle a_1, a_2, a_3 \rangle$
$= \langle 9, 4, 6 \rangle$:

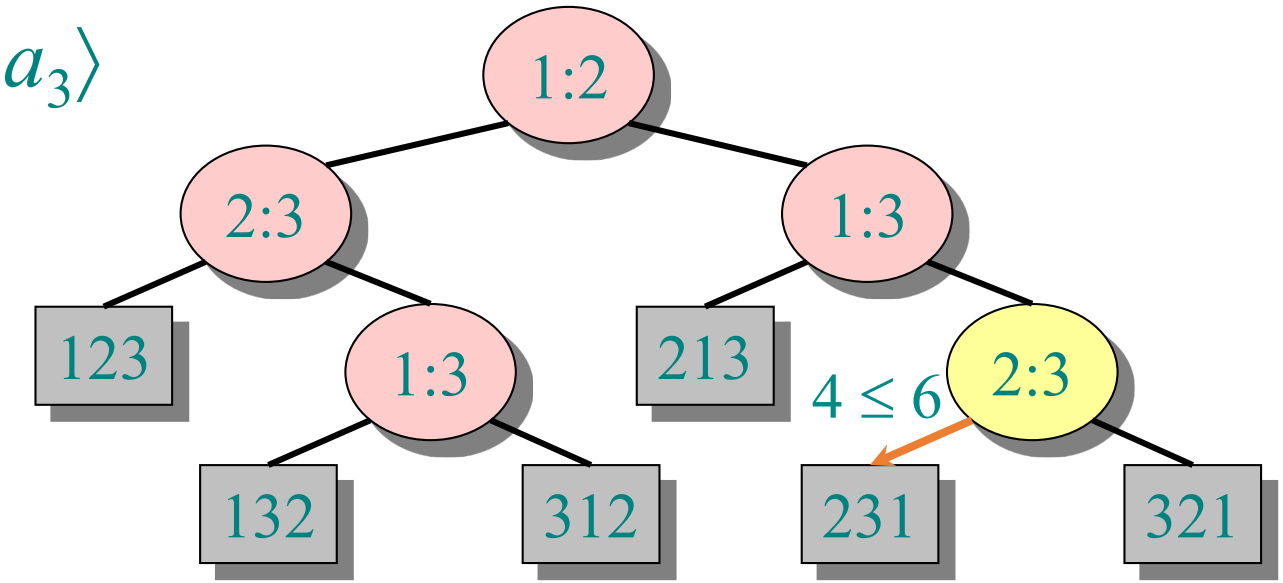


Each internal node is labeled $i{:}j$ for $i, j \in \{1, 2, \ldots, n\}$.

- The left subtree shows subsequent comparisons if $a_i < a_j$.
- The right subtree shows subsequent comparisons if $a_i \geq a_j$.

Decision-tree example

Sort $\langle a_1, a_2, a_3 \rangle$
$= \langle 9, 4, 6 \rangle$:

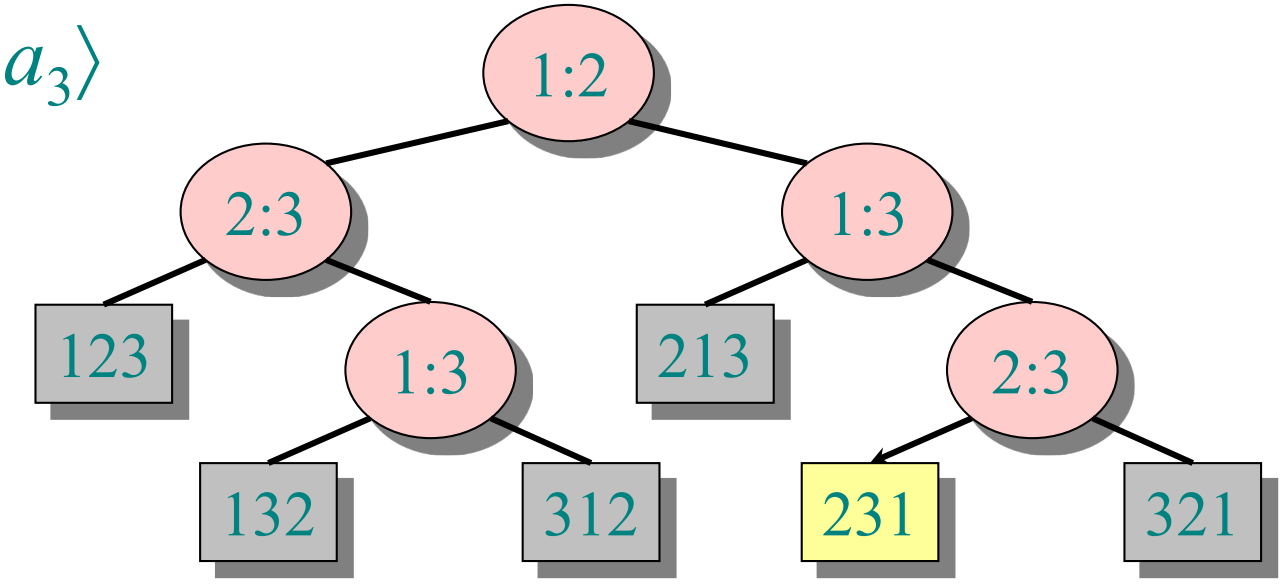


Each internal node is labeled $i{:}j$ for $i, j \in \{1, 2, \ldots, n\}$.

- The left subtree shows subsequent comparisons if $a_i < a_j$.
- The right subtree shows subsequent comparisons if $a_i \geq a_j$.

Decision-tree example

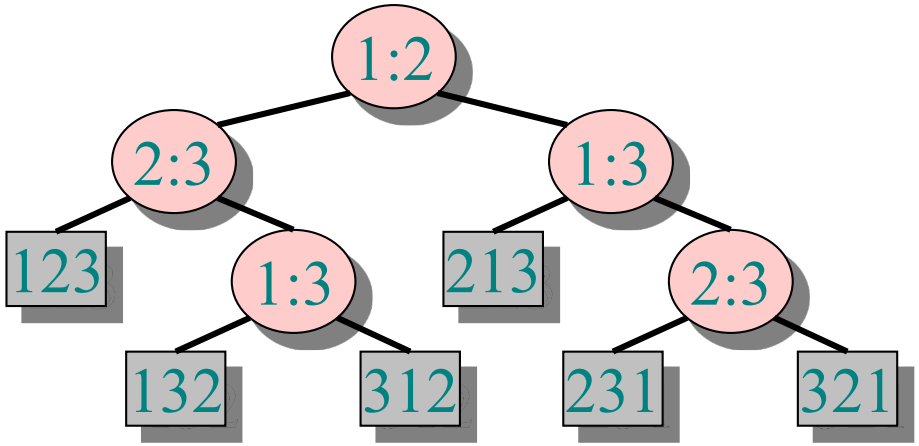
Sort $\langle a_1, a_2, a_3 \rangle$
$= \langle\ 9, 4, 6\ \rangle$:



$4 \le 6 \le 9$

Each leaf contains a permutation $\langle \pi(1), \pi(2), \ldots, \pi(n) \rangle$ to indicate that the ordering $a_{\pi(1)} \le a_{\pi(2)} \le \Lambda \le a_{\pi(n)}$ has been established.

Decision-tree model



A decision tree can model the execution of any comparison sort:

- One tree for each input size n.
- View the algorithm as splitting whenever it compares two elements.
- The tree contains the comparisons along all possible instruction traces.
- The running time of the algorithm = the length of the path taken.
- Worst-case running time = height of tree.

Lower bound for decision-tree sorting

- **Theorem.** Any decision tree that can sort n elements must have height $\Omega(n \lg n)$.
- *Proof.* The tree must contain $\geq n!$ leaves, since there are $n!$ possible permutations. A height-h binary tree has $\leq 2^h$ leaves. Thus, $n! \leq 2^h$.

$$
\begin{aligned}
\therefore \quad h &\geq \lg(n!) && \text{(lg is mono. increasing)} \\
&\geq \lg((n/e)^n) && \text{(Stirling's formula)} \\
&= n \lg n - n \lg e \\
&= \Omega(n \lg n). \quad \blacksquare
\end{aligned}
$$

Lower bound for comparison sorting

Corollary. Heapsort and merge sort are asymptotically optimal comparison sorting algorithms.

Can we sort faster?

- $\Omega(n \log n)$ is the lower bound for sorting.
- Can we break the lower bound?

- The sorting lower bound assumes we only compare elements.
- If we do more than comparison, we may be able to break the lower bound!

Sorting in linear time

- Assume there are k different elements. We can perform counting sort.

Counting sort: No comparisons between elements.

- ***Input***: $A[1 \,.\,.\, n]$, where $A[j] \in \{1, 2, \ldots, k\}$.
- ***Output***: $B[1 \,.\,.\, n]$, sorted.
- ***Auxiliary storage***: $C[1 \,.\,.\, k]$.

3 steps in counting sort

1. Set C[i] be the number of elements equal i for i=1, 2, …, k.

2. Set C[i] be the number of elements smaller than or equal i for i=1, 2, …, k.

3. Move elements equal i to B[C[i-1]+1..C[i]] for i=1, 2, …, k.

Counting sort

Set C[i] be the number of elements equal i.

for $i \leftarrow 1$ **to** k
  do $C[i] \leftarrow 0$
for $j \leftarrow 1$ **to** n
  do $C[A[j]] \leftarrow C[A[j]] + 1$ $\triangleright$ $C[i] = |\{\text{key} = i\}|$

Set C[i] be the number of elements smaller than or equal i.

for $i \leftarrow 2$ **to** k
  do $C[i] \leftarrow C[i] + C[i-1]$ $\triangleright$ $C[i] = |\{\text{key} \leq i\}|$

Move elements equal i to B[C[i-1]+1..C[i]].

for $j \leftarrow n$ **downto** 1
  do $B[C[A[j]]] \leftarrow \mathrm{A}[j]$
    $C[A[j]] \leftarrow C[A[j]] - 1$

Counting-sort example

	1	2	3	4	5
A:	4	1	3	4	3
B:					

	1	2	3	4
C:				

Phase 1 (I)

	1	2	3	4	5
A:	4	1	3	4	3

	1	2	3	4
C:	0	0	0	0

B:					

for $i \leftarrow 1$ **to** k
    do $C[i] \leftarrow 0$

Phase 1 (II)

	1	2	3	4	5
A:	4	1	3	4	3

	1	2	3	4
C:	1	0	2	2

B:					

for $j \leftarrow 1$ **to** n

  do $C[A[j]] \leftarrow C[A[j]] + 1$ $\quad \triangleright\ C[i] = |\{\text{key} = i\}|$

Phase 2

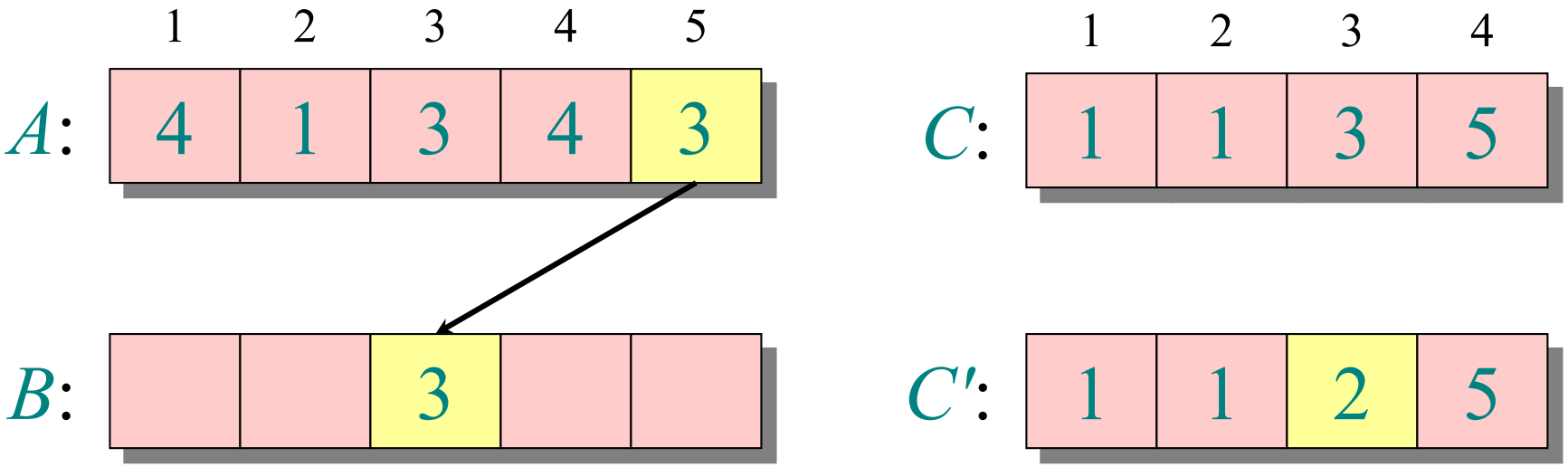
	1	2	3	4	5
A:	4	1	3	4	3
B:					

	1	2	3	4
C:	1	0	2	2
C':	1	1	3	5

for $i \leftarrow 2$ **to** k

  do $C[i] \leftarrow C[i] + C[i-1]$ $\quad \triangleright\ C[i] = |\{\text{key} \le i\}|$

Phase 3

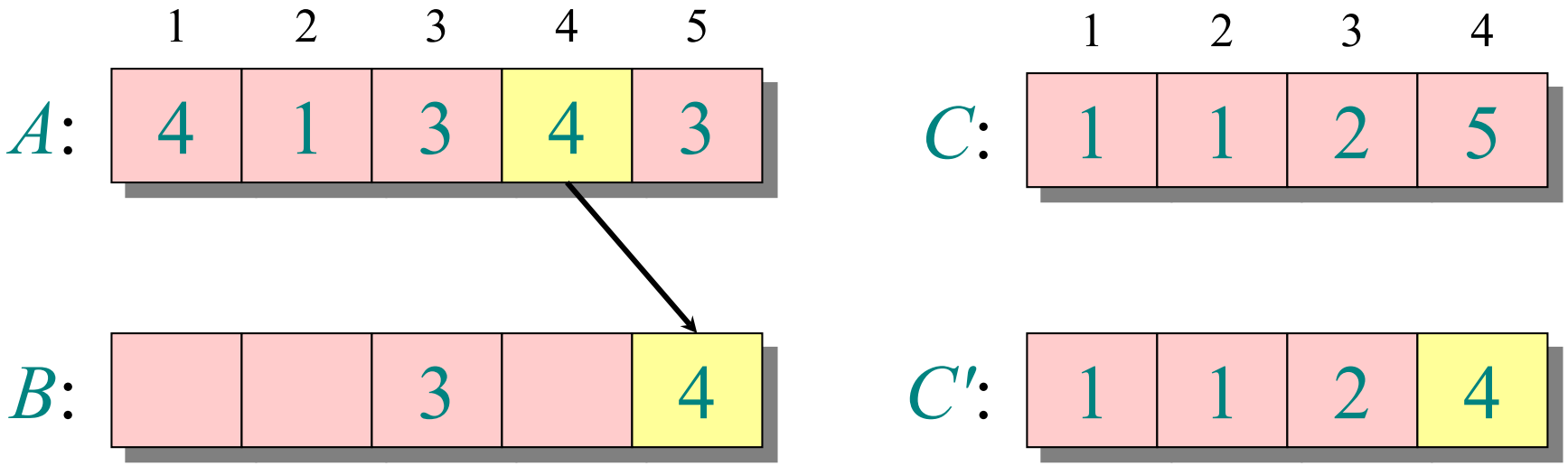


for $j \leftarrow n$ **downto** 1

  do $B[C[A[j]]] \leftarrow A[j]$

    $C[A[j]] \leftarrow C[A[j]] - 1$

Phase 3

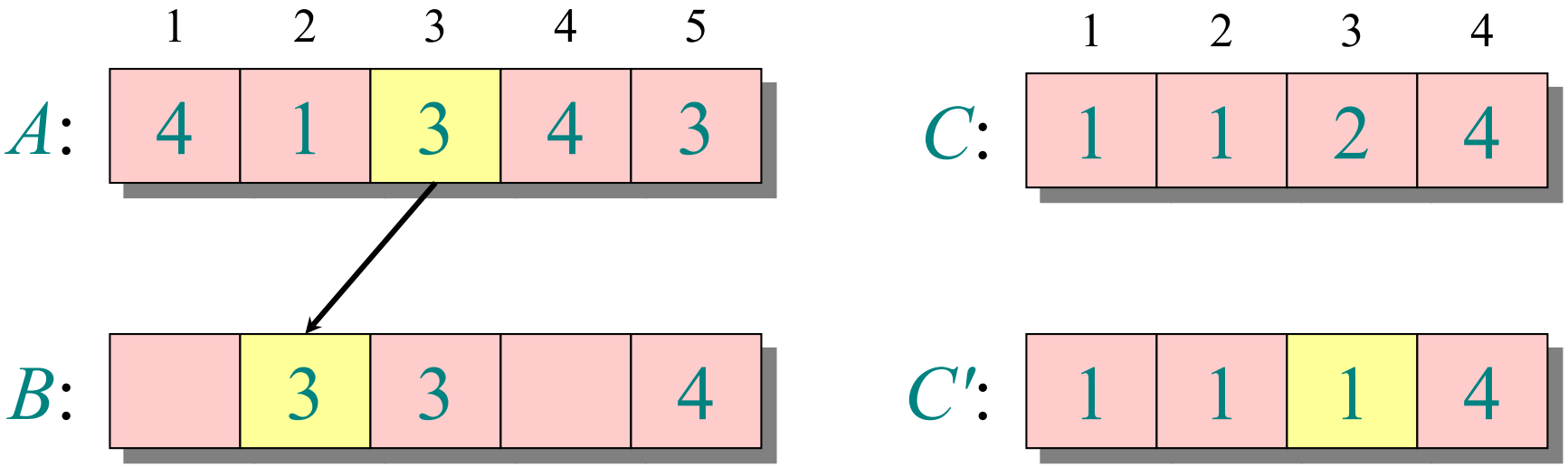


for $j \leftarrow n$ **downto** 1

  do $B[C[A[j]]] \leftarrow A[j]$

    $C[A[j]] \leftarrow C[A[j]] - 1$

Phase 3



for $j \leftarrow n$ **downto** 1

  do $B[C[A[j]]] \leftarrow A[j]$

    $C[A[j]] \leftarrow C[A[j]] - 1$

Phase 3

	1	2	3	4	5
A:	4	1	3	4	3
B:	1	3	3		4

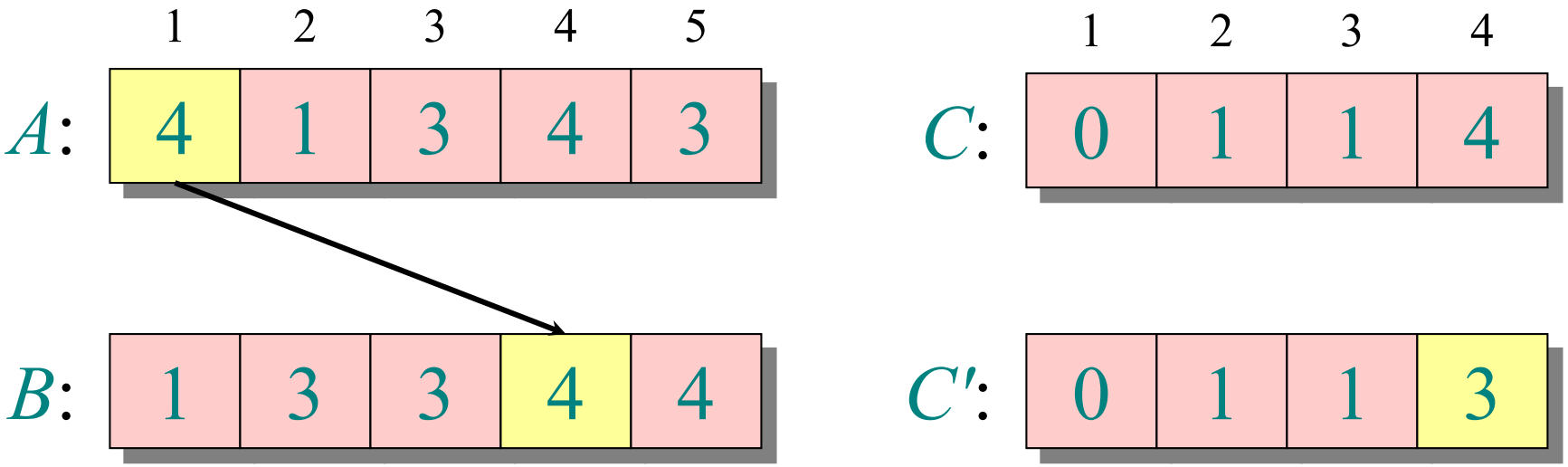
	1	2	3	4
C:	1	1	1	4
C':	0	1	1	4

for $j \leftarrow n$ **downto** 1

  do $B[C[A[j]]] \leftarrow A[j]$

    $C[A[j]] \leftarrow C[A[j]] - 1$

Phase 3



for $j \leftarrow n$ **downto** 1

  do $B[C[A[j]]] \leftarrow A[j]$

    $C[A[j]] \leftarrow C[A[j]] - 1$

Analysis

$\Theta(k)$ $\left\{ \begin{array}{l} \textbf{for } i \leftarrow 1 \textbf{ to } k \\ \quad \textbf{do } C[i] \leftarrow 0 \end{array} \right.$

$\Theta(n)$ $\left\{ \begin{array}{l} \textbf{for } j \leftarrow 1 \textbf{ to } n \\ \quad \textbf{do } C[A[j]] \leftarrow C[A[j]] + 1 \end{array} \right.$

$\Theta(k)$ $\left\{ \begin{array}{l} \textbf{for } i \leftarrow 2 \textbf{ to } k \\ \quad \textbf{do } C[i] \leftarrow C[i] + C[i-1] \end{array} \right.$

$\Theta(n)$ $\left\{ \begin{array}{l} \textbf{for } j \leftarrow n \textbf{ downto } 1 \\ \quad \textbf{do } B[C[A[j]]] \leftarrow \mathrm{A}[j] \\ \qquad\quad C[A[j]] \leftarrow C[A[j]] - 1 \end{array} \right.$

$\Theta(n + k)$

Running time

If $k = O(n)$, then counting sort takes $\Theta(n)$ time.

- But, sorting takes $\Omega(n \lg n)$ time!
- Where's the fallacy?

Answer:

- ***Comparison sorting*** takes $\Omega(n \lg n)$ time.
- Counting sort is not a ***comparison sort***.
- In fact, not a single comparison between elements occurs!

Is counting sort a stable sorting?

- Yes.

Radix sort

- ***Origin***: Herman Hollerith's card-sorting machine for the 1890 U.S. Census.
- Digit-by-digit sort.
- Hollerith's original (bad) idea: sort on most-significant digit first.
- Good idea: Sort on ***least-significant digit first*** with auxiliary ***stable*** sort.

Operation of radix sort

Suppose there are T digits.

for $t \leftarrow 1$ **to** T

do sort the tth least significant digit by a **stable** sorting algorithm;

3	2	9	7	2	0	7	2	0	3	2	9
4	5	7	3	5	5	3	2	9	3	5	5
6	5	7	4	3	6	4	3	6	4	3	6
8	3	9	4	5	7	8	3	9	4	5	7
4	3	6	6	5	7	3	5	5	6	5	7
7	2	0	3	2	9	4	5	7	7	2	0
3	5	5	8	3	9	6	5	7	8	3	9

Example: We can use counting sort as the stable sorting algorithm.

Correctness of radix sort

- Proof by induction.
- P(t): The numbers are sorted by their low-order t digits.

- Base case:
  - The radix sort algorithm sorts the 1st digit by stable sorting algorithm.
  - Hence, P(1) is true.
- Inductive case:
  - Assume P(t-1) is true.
  - We need to show P(t) is true.

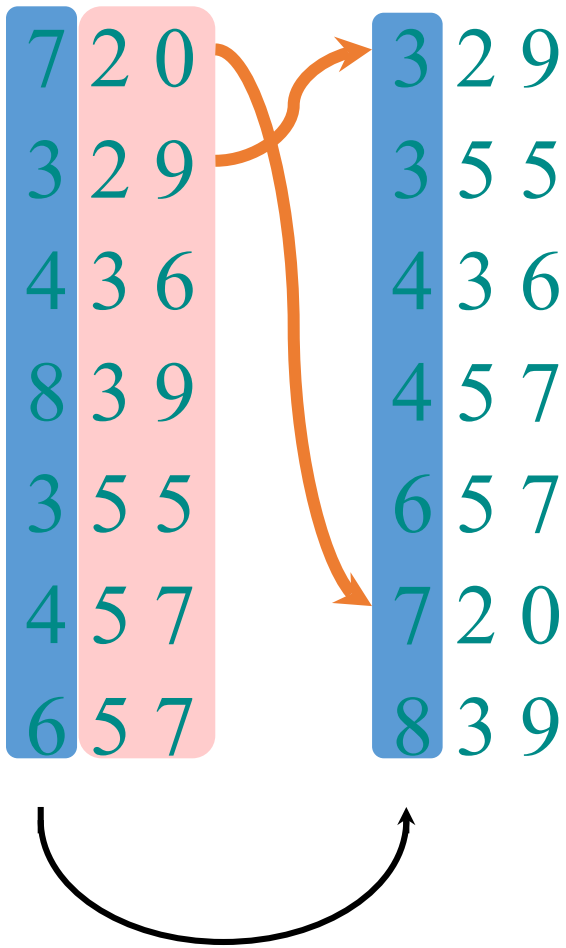
Suppose the numbers are sorted by their low-order t-1 digits (e.g. t=3)

7	2	0	3	2	9
3	2	9	3	5	5
4	3	6	4	3	6
8	3	9	4	5	7
3	5	5	6	5	7
4	5	7	7	2	0
6	5	7	8	3	9

Correctness of radix sort

Induction on digit position

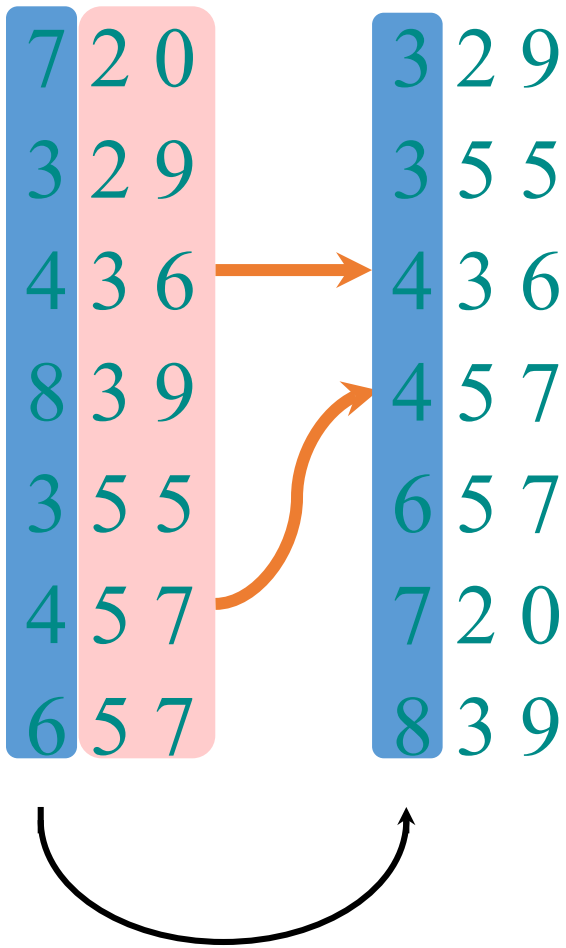
- Assume that the numbers are sorted by their low-order $t - 1$ digits.
- Sort on digit t
  - Two numbers that differ in digit t are correctly sorted.



Correctness of radix sort

Induction on digit position

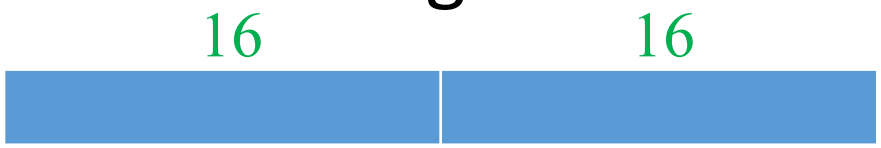
- Assume that the numbers are sorted by their low-order $t - 1$ digits.
- Sort on digit t
  - Two numbers that differ in digit t are correctly sorted.
  - Two numbers equal in digit t are put in the same order as the input $\Rightarrow$ correct order.



Analysis of radix sort

- Suppose counting sort is used as the auxiliary stable sort.
- Sort n computer words of b bits each.
- Each word can be viewed as having b/r base-2^r digits.
- Example: 32-bit word

- $r = 8 \Rightarrow b/r = 4$ passes of counting sort on base-2^8 digits; or $r = 16 \Rightarrow b/r = 2$ passes of counting sort on base-2^{16} digits.



- To optimize the running time, how many passes (b/r) should we make?

Analysis (continued)

- **Recall:** Counting sort takes $\Theta(n + k)$ time to sort n numbers in the range from 1 to k.
- If each b-bit word is broken into r-bit pieces, each pass of counting sort takes $\Theta(n + 2^r)$ time. Since there are b/r passes, we have:
  - $T(n, b) = \Theta(\frac{b}{r}(n + 2^r))$

Choose r to minimize $T(n, b)$:

- Increasing r means fewer passes, but as $r > \lg n$, the time grows exponentially.

Choosing r

$$T(n,b) = \Theta\left(\frac{b}{r}\left(n + 2^r\right)\right)$$

Choose r that minimizes $T(n,b)$ (by differentiation).

The optimal r is close to $\lg n$.
(Optimal r is slightly smaller than $\lg n$.)

Choosing $r = \lg n$ implies $T(n,b) = \Theta(bn/\lg n)$.

- For numbers in the range from 0 to $n^d - 1$, we have $b = d \lg n \Rightarrow$ radix sort runs in $\Theta(d\,n)$ time.

When should we use radix sort?

- Radix sort is simple to code and maintain.
- In practice, radix sort is fast when the number of passes is small.

Example (32-bit numbers):

- At most 3 passes when sorting ≥ 2000 numbers.
- Merge sort and quicksort do at least $\lceil \lg 2000 \rceil = 11$ passes.

- **Downside:** Unlike quicksort, radix sort displays little locality of reference, and thus a well-tuned quicksort fares better on modern processors, which feature steep memory hierarchies.

Conclusion

- This lecture covers the analysis of sorting.
- We describe the lower bound for comparison sort, which is $\Omega(n \lg n)$.
- We also discuss linear time sorting algorithms:
  - Counting sort and radix sort.

- When you design an algorithm to sort elements,
  - Select suitable sorting algorithms depends on the properties of the elements.
  - A number of issues:
    - Memory issue: InPlace sort
    - Need to preserve the original order of equal elements: Stable sort
    - Average case fast: QuickSort
    - Worst case fast (comparison model): worst case O(n log n)-time sort
    - Worst case fast (non-comparison model): radix sort

Acknowledgement

- The slides are modified from
  - the slides from Prof. Erik D. Demaine and Prof. Charles E. Leiserson
  - the slides from Prof. Lee Wee Sun